

ALGORITMO y ESTRUCTURA DE DATOS II





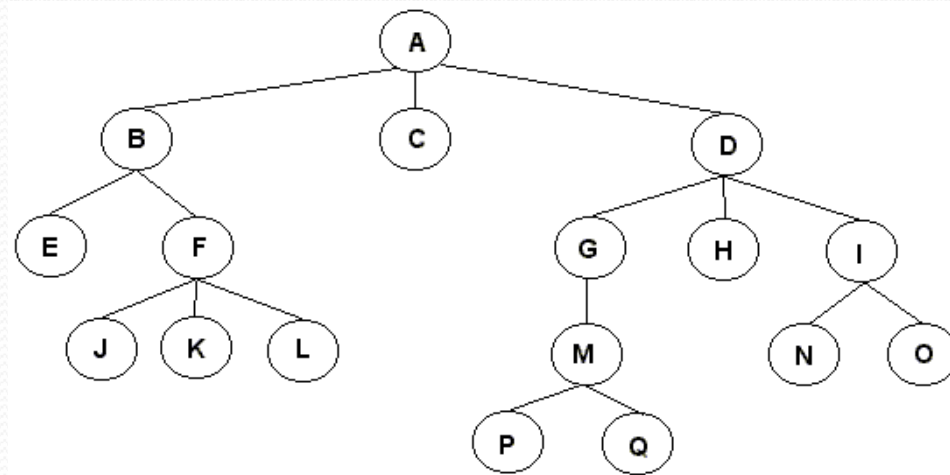
Arboles

Esta estructura conocida como árbol, es apropiada para la representación de relaciones jerárquicas.

Ej: Relación que existen entre los miembros de una familia.
Jerarquía entre los que trabajan en una misma empresa.
Jerarquía entre los componentes de un programa.

Arboles

- Son estructura que permiten representar datos que tienen enlaces jerárquicos entre ellos.



Arboles: Definición.

Un árbol es una estructura de datos que satisface tres propiedades:

- Cada elemento del árbol se relaciona con cero o más elementos a quienes llama hijos.
- Si el árbol no está vacío, hay un único elemento al cual se llama raíz y que no tiene padre (predecesor), es decir, no es hijo de ningún otro.
- Todo otro elemento del árbol posee un único padre y es un descendiente (hijo del hijo del hijo, etc.) de la raíz.

Representación grafica de Árbol

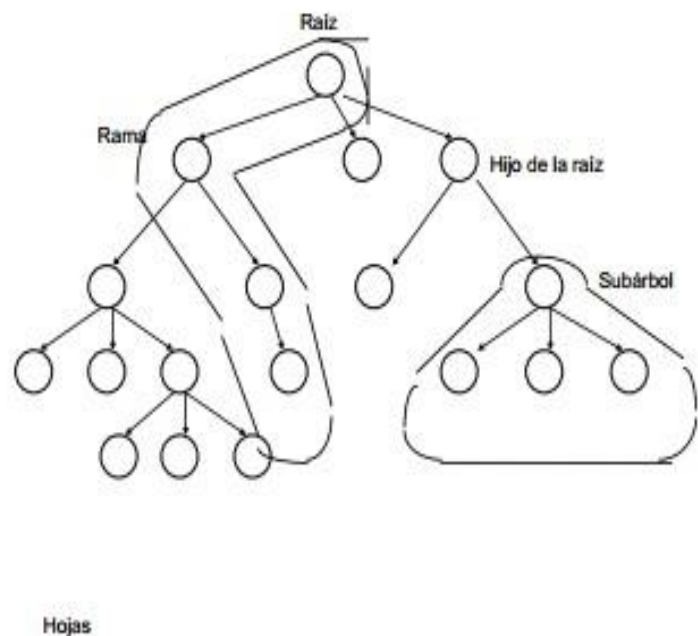


Fig.1

Gráficamente, los árboles se representan mediante esquemas como el que muestra la Fig.1, donde pueden verse: líneas que conectan a cada elemento con sus hijos.

La raíz aparece en el tope del árbol, y a diferencia de los árboles que conocemos habitualmente, estos crecen hacia abajo.

Cada elemento del árbol se denomina nodo y los nodos que no tienen hijos son las hojas del árbol.

Un camino en el árbol es una secuencia lineal de elementos.

Una rama en un árbol es un camino que se extiende desde la raíz hasta una hoja.

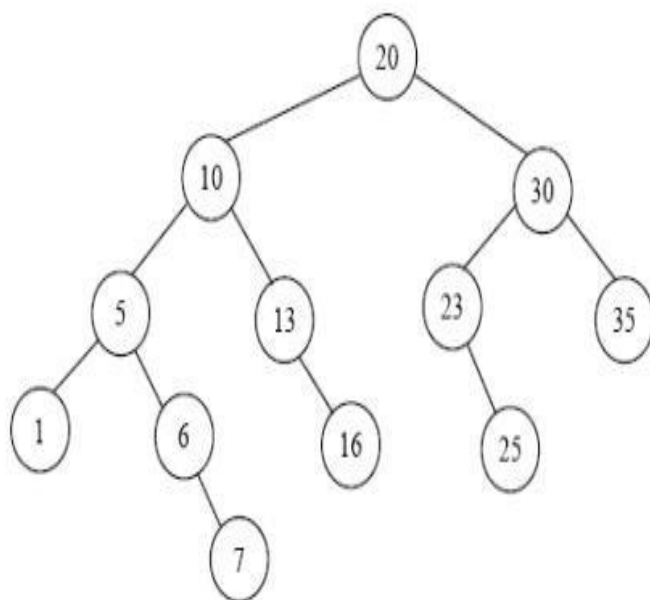
Un subárbol de un árbol es un nodo con todos sus descendientes.

El nivel de un nodo en el árbol es la longitud del camino desde la raíz al nodo específico (comienza en cero)

La altura de un árbol equivale al nivel mas alto mas uno.

Ejercicio N° 1

Considere el árbol siguiente:



1. ¿Cuál es su altura?
2. ¿Está el árbol equilibrado? ¿Por Qué?
3. Listar todos los nodos hoja.
4. ¿Cuál es el predecesor inmediato (padre) del nodo 6?
5. Listar los hijos del nodo 10
6. Listar los sucesores del nodo 10

Árbol binario:

- Un árbol donde cada nodo no tiene más de dos hijos se denomina árbol binario.
- Se dice que cada nodo tiene a lo sumo: un hijo izquierdo y un hijo derecho.

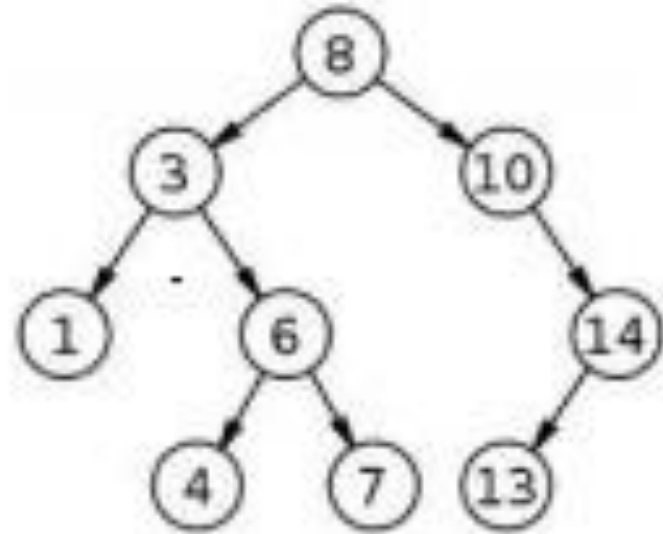


Fig.2



Árbol binario ordenado

Un árbol binario se dice que está ordenado cuando cada nodo del árbol es menor que los elementos de su subárbol izquierdo y mayor o igual que los de su subárbol derecho.

Como puede verse, una pequeña variante del método de búsqueda dicotómico permite hallar un elemento en un árbol binario ordenado.

El elemento buscado se compara con la raíz y si es menor se repite el proceso comparando contra el hijo izquierdo en caso contrario continuamos por la derecha.

El proceso termina cuando encontramos el elemento o cuando se termina la rama.

Representación de árbol binario

Para el manejo de árboles utilizaremos una estructura basada en punteros como lo hicimos en Lista. Podemos utilizar la siguiente representación:

type

TipoElem : ... ;

ArbolBinario = ^Nodo;

Nodo = record

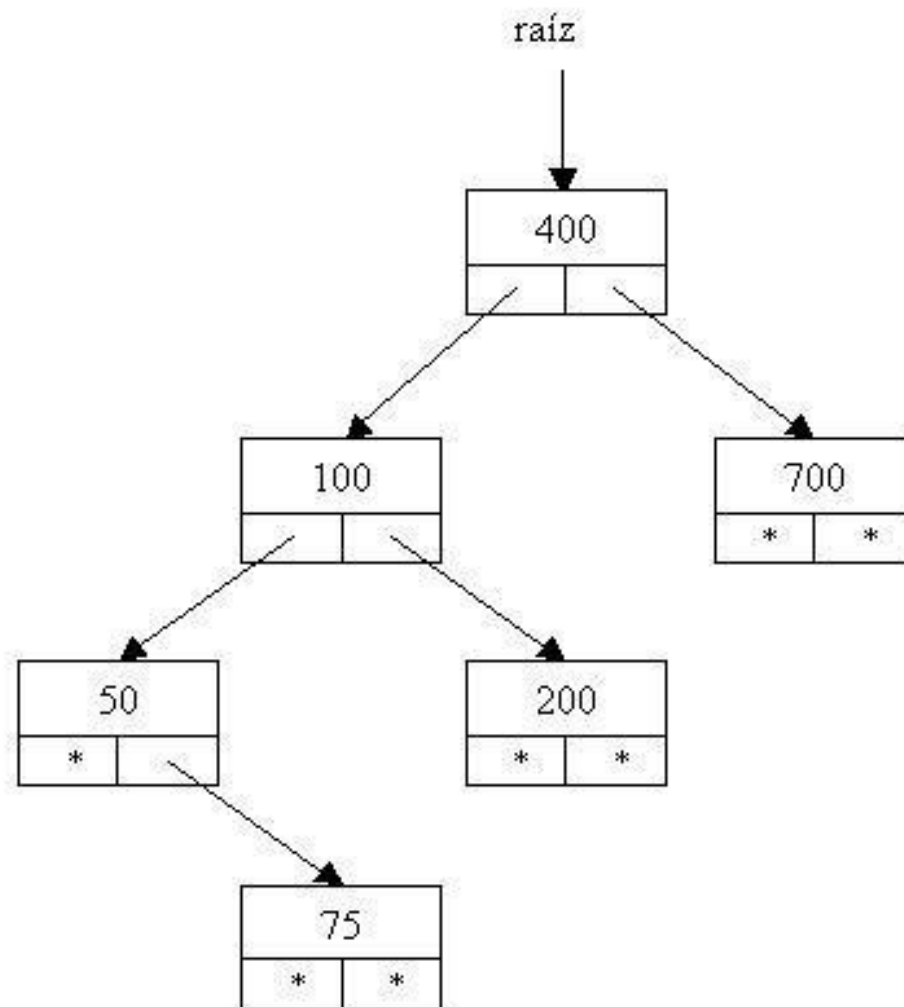
elem : TipoElem;

izq, der : ArbolBinario;

end;

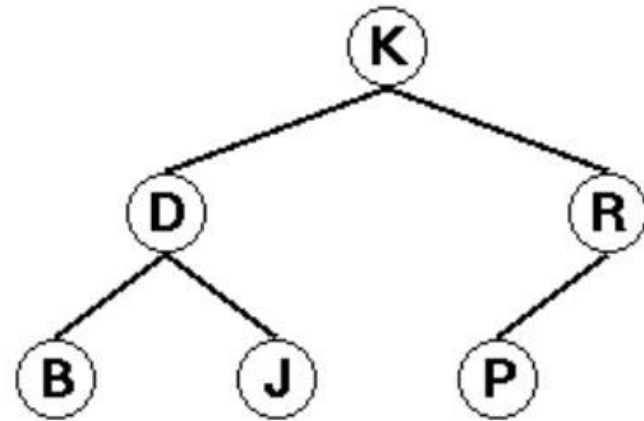
var

arbol : ArbolBinario;



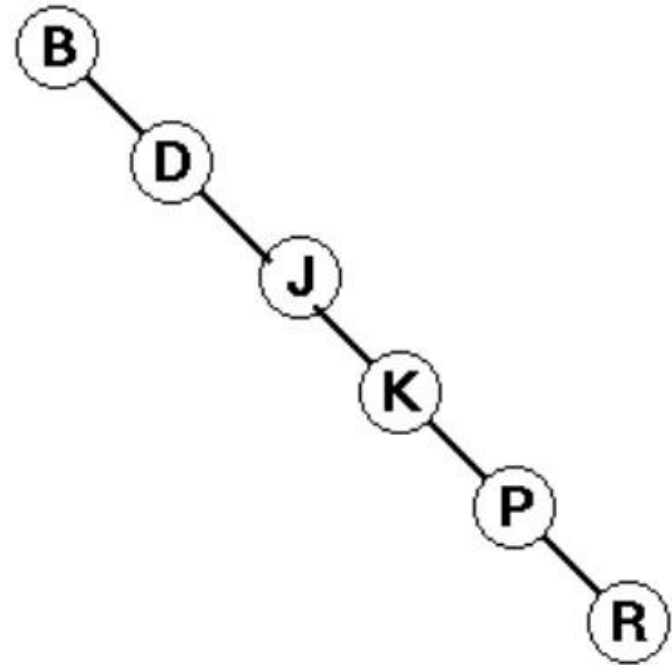
Árboles equilibrados

- Un árbol equilibrado es aquel en el que las ramas izquierda y derecha de cada nodo tienen aproximadamente la misma altura.
- El árbol lleno sería el árbol equilibrado perfecto con todos los nodos con subárboles izq y dech de la misma altura
- Las búsquedas son más eficientes cuando el árbol está equilibrado



Árboles “degenerados”

- El caso peor ocurre cuando el árbol está “degenerado”, es decir, sigue siendo un árbol pero su estructura es equivalente a una lista enlazada (todos los nodos sólo tienen un hijo)



Dibujar el árbol binario de búsqueda que se construye cuando las letras y los números se insertan en el orden dado

- a) M, Y, T, E, R
- b) R, E, M, Y, T
- c) T, Y, M, E, R
- d) C, O, R, N, F, L, A, K, E, S
- e) 1, 2, 3, 4, 5, 6, 7
- f) 4, 2, 1, 3, 6, 5, 7
- g) 1, 6, 7, 2, 4, 3, 5
- h) 7, 25, 27, 15, 23, 19, 14, 29, 10, 50, 18, 22, 46, 17, 70, 33, 58.
- i) C, S, D, T, A, M, P, I, B, W, N, G, U, R, K, E, H, O, L, J
- j) 6, 21, 39, 12, 33, 15, 35, 7, 26, 18, 22, 5, 1, 8.

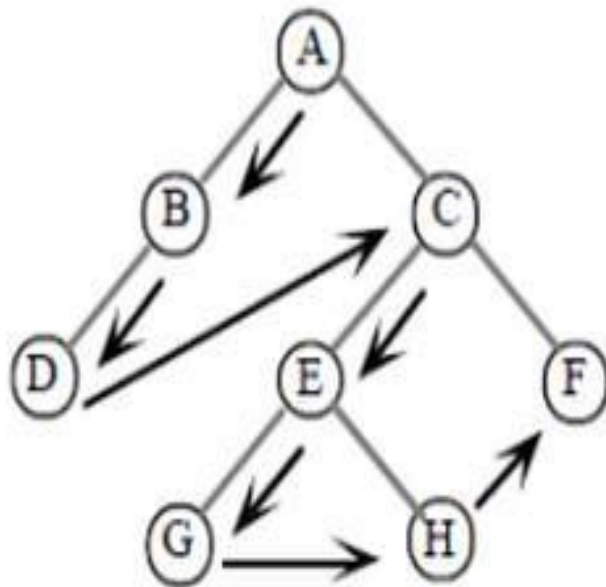
Recorrido de un Árbol binario

- Recorrer un árbol consiste en acceder una sola vez a todos sus nodos. Esta operación es básica en el tratamiento de árboles y nos permite, por ejemplo, imprimir toda la información almacenada en el árbol, o bien eliminar toda esta información o, si tenemos un árbol con tipo base numérico, sumar todos los valores...

Tipos de recorrido

- Preorden : raiz - subarbolizq - subarbolder
- En orden: subarbolizq – raiz - subarbolder
- Postorden: subarbolizq - subarbolder - raiz

Preorden : raiz - subarbolizq - subarbolder



A - B - D - C - E - G - H - F



```
Procedure preorden(p:arbolbinario);
```

```
Begin
```

```
  If (p <> nil) then
```

```
    Begin
```

```
      Write (p ^.elem);
```

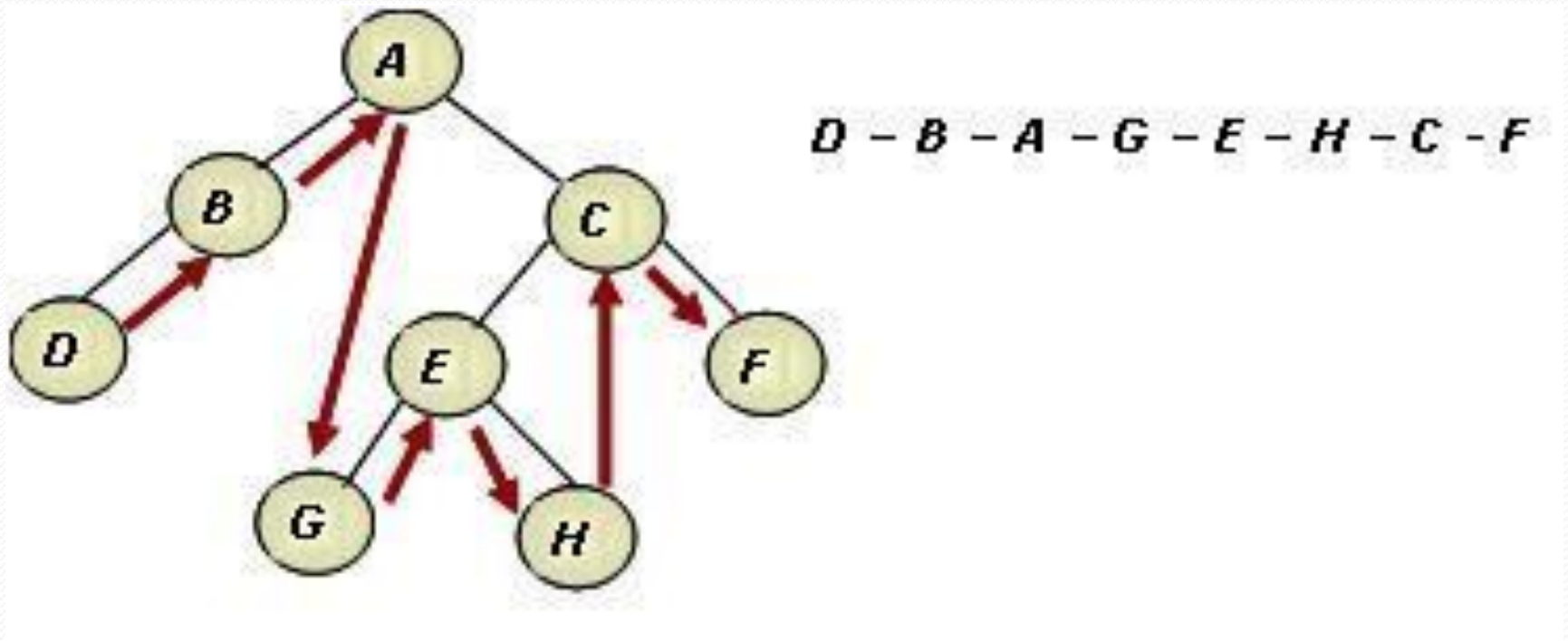
```
      Preorden(p^.izq);
```

```
      Preorden (p^.der);
```

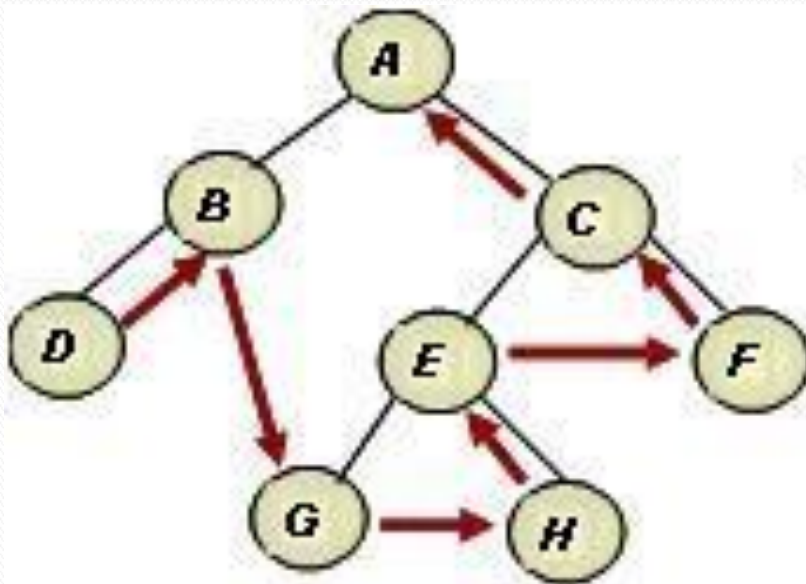
```
    End;
```

```
End;
```

En orden: subarbolizq – raiz - subarbolder



Postorden: subarbolizq - subarbolder - raiz



D - B - G - H - E - F - C - A

- 
- Realizar recorridos Inorden, Preorden y Postorden de los árbol anteriores y mostrar la secuencia de letras o números que resultan en cada caso.
 - Ejercicio para subir como tarea en el aula

15, 3, 55, 12, 11, 14, 67, 21, 27, 44, 22, 1, 5, 33, 8 y 34

Dibujar el árbol binario y realizar los tres recorridos

1. Inteligencia Artificial (IA)

Árboles de decisión → clasificación y predicción

Random Forest → ensambles para mejorar precisión

Árboles de búsqueda heurística → A* y algoritmos de planificación

Árboles de juego → minimax, poda alfa-beta

2. Sistemas y estructuras de datos

Árbol binario → base para búsquedas y recorridos

Árbol balanceado (AVL, Red-Black) → eficiencia en inserción/borrado

Árboles de segmentos y Fenwick → optimización en consultas de rangos

Árboles de sufijos → procesamiento de cadenas

3. Bases de datos

B-Tree y B+Tree → índices para acceso rápido

Árboles de búsqueda → organización de registros

Trie → almacenamiento eficiente de claves

4. Compiladores y lenguajes

Árbol sintáctico → representación de la estructura del código

Árbol abstracto de sintaxis (AST) → análisis semántico y optimización

5. Redes y sistemas distribuidos

Árboles de enrutamiento → protocolos jerárquicos

Árboles de expansión mínima → optimización de topologías

Árboles multicast → distribución eficiente de datos

6. Gráficos y visualización

Quadtrees y Octrees → representación espacial

Árboles de escena → organización de objetos en gráficos 3D

E
j
e
m
p
l
o
s

d
e

u
s
o



Bibliografía

- **“Introduction to Algorithms”** – Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein.
Referencia fundamental en algoritmos y estructuras de datos, incluye árboles binarios, AVL, Red-Black Trees y más.
- **“Data Structures and Algorithm Analysis in C++”** – Mark Allen Weiss.
Explica árboles binarios de búsqueda, árboles balanceados y aplicaciones prácticas.
- **“Algorithms”** – Robert Sedgewick y Kevin Wayne.
Muy usado en cursos universitarios, con ejemplos claros de árboles binarios y su implementación.
- **“Fundamentals of Data Structures”** – Ellis Horowitz, Sartaj Sahni.
Texto clásico que aborda árboles binarios y otras estructuras.

Recursos en línea

- **GeeksforGeeks – Binary Trees**
Explicaciones prácticas, ejemplos en varios lenguajes de programación.
- **Tutorialspoint – Data Structures Trees**
Introducción sencilla y ejemplos básicos.
- **Coursera / edX cursos de algoritmos y estructuras de datos**
Muchos cursos incluyen módulos sobre árboles binarios.